

回顾

可迭代对象、迭代器和生成器

■ 可迭代对象

```
1 1、定义：可通过for循环迭代读取数据供我们使用的对象
2 2、本质：可迭代对象通过__iter__()方法向我们提供一个迭代器
3 3、示例
4 name_list = ['紫衫龙王', '金毛狮王', '白眉鹰王', '青翼蝠王']
5 name_iterator = name_list.__iter__() # 提供该可迭代对象(name_list)的迭代器
6 for name in name_iterator:
7     print(name)
8
9 4、__iter__()函数与__next__()函数的作用是什么？
10 1、__iter__()函数：获取可迭代对象的迭代器
11 2、__next__()函数：对迭代器不断使用__next__()函数来获取下一条数据，完成之后再次调用会抛出
    StopIteration异常
```

■ 迭代器

```
1 一个实现了__iter__()方法和__next__()方法的对象，就是迭代器
```

■ 生成器

```
1 1、函数中有yield关键字，就称为生成器
2 2、yield作用
3 1、保存当前运行状态(断点)，然后暂停执行，即将生成器（函数）挂起
4 2、将yield关键字后面表达式的值作为返回值返回，此时可以理解为起到了return的作用
5 3、如何启动生成器
6 next()：单纯的获取生成器中的一个值
7 4、示例
8 def f1():
9     for i in range(3):
10         yield i
11 g = f1()
12 print(next(g))
```

并发和并行

```
1 # 1. 并发
2 同时处理多个任务,内核在任务间不断地切换,达到好像多个任务同时被执行的效果,实际上每一时刻只有1个任务在占有CPU
3
4 # 2. 并行
5 一起执行,多个任务利用计算机的多核资源同时执行,此时多个任务之间是并行关系
6
7 # 3. 示例
8 在计算机系统中,并行和并发是同时存在的
9 计算机 : 2核
10 任务   : QQ、微信、爱奇艺、英雄联盟、Chrome
11 并发   : 这5个任务间存在着并发关系
12 并行   : 第1核(QQ)、第2核(微信)
```

进程、线程、协程

```
1 # 1. 进程
2 程序的运行过程,是CPU分配资源的最小单位
3
4 # 2. 线程
5 最小执行单位,是系统分配内核的最小单位,由所属进程管理,一个进程可有多个线程
6
7 # 3. 协程
8 1、定义 : 微线程,协程允许在不同的位置暂停或者开始执行,简单来说,协程就是可以暂停执行的函数
9 2、原理 : 记录一个函数栈的上下文,进行协程的切换调度,当一个函数暂停执行时,会将上下文栈帧保存起来
10 3、本质 : 单线程,资源消耗小,无须线程间切换的开销,无需加互斥锁
11 4、协程模块 : greenlet、gevent、... ..
```

基于协程的多任务并发?

```
1 |
```

```
1 |
```

Python中的那些锁

■ GIL锁

```
1 # 1. GIL是什么? - CPython
2 全局解释器锁,限制多线程同时执行,保证同一时间内只有一个线程在执行
3
4 # 2. 作用
5 同一进程中线程是数据共享,当各个线程访问数据资源时会出现竞争状态,即数据可能会同时被多个线程占用,造成数据混乱,这就是线程的不安全。而解决多线程之间数据完整性和状态同步最简单的方式就是加锁。GIL能限制多线程同时执行,保证同一时间内只有一个线程在执行
6
7 # 3. 影响
8 影响多线程效率
9
10 # 4. 如何避免?
11 1、用多进程 代替 多线程
12 2、更换解释器
```

▪ 互斥锁

```
1 # 1. 问题原因
2 多个线程共享数据时,如果数据不进行保护,则可能出现数据不一致现象
3 # 2. 解决方案
4 使用一把锁把代码保护起来,以牺牲性能换取代码的安全性
5 # 3. 示例
```

▪ 死锁

```
1 多个进程/线程在执行过程中,因争夺资源而造成的一种互相等待的现象,若无外力作用,它们都无法进行下去,此时称系统处于死锁状态
```

产生死锁

```
1 |
```

避免死锁

```
1 |
```

MySQL数据库

▪ 数据库的三范式

```
1 # 1. 三范式
2 1NF:字段不可分
3 2NF:有主键,非主键字段依赖主键
4 3NF:非主键字段不能相互依赖
5
6 # 2. 解释
7 1NF:原子性 字段不可再分,否则就不是关系数据库
8 2NF:唯一性 一个表只说明一个事物
9 3NF:每列都与主键有直接关系,不存在传递依赖
10
```

```

11 # 3. 解释
12 1NF:
13 符合1NF: 用户ID 用户名 密码 姓名 电话
14 不符合1NF: 用户ID 用户名 密码 用户信息(包含姓名和电话)
15
16 2NF:
17 第二范式是在第一范式的基础之上建立起来的,满足第二范式前提必须满足第一范式,要求数据库表中的每个实例
    或行都能被唯一区分,建立主键
18
19 3NF:
20 1、满足第三范式必须先满足第二范式
21 2、要求: 属性不依赖于其他非主属性, 简而言之为如果表的信息能被推导出来,就不能单独设计一个字段来存放
22 3、如果一个实体中出现其他实体的非主属性, 可以将这两个实体用外键关联, 而不是将另一张表的非主属性直
    接写在当前表中
23
24 表1:
25 商品名称 价格 描述 有效期 分类 分类描述
26 雪碧 3 甜 2020 酒水饮料 碳酸饮料
27
28 重新设计后:
29 表1:商品信息表(product) 表2:分类信息表(product_type)
30 商品ID 商品名称 价格 描述 有效期 分类ID 分类 分类描述
31 1 雪碧 3 甜 2020 1 酒水饮料 碳酸饮料
32
33 表3:中间表(product_type) - 做外键关联
34 商品ID 分类ID
35 1 1

```

■ 关联查询

```

1 1、内连接(inner join)
2 2、左外连接(left join):以左表为主显示查询结果
3 3、右外连接(right join):以右表为主显示查询结果
4 select xxx from 表1 inner|left|right join 表2 on 条件;
5 4、问题: 查询商品的名称(name)、价格(price)以及所属分类(type)???

```

■ 子查询

```

1 把外层命令的查询结果作为内层查询的查询条件,就是一条查询语句中又嵌套了查询语句

```

■ 索引的优缺点及建立原则

```

1 # 1. 优点
2 1、提高检索速度
3 2、唯一性索引可保证数据库表中每一行数据的唯一性
4 3、使用分组和排序子句进行数据检索时, 可以显著减少查询中分组和排序的时间
5
6 # 2. 缺点
7 1、创建索引和维护索引需要耗费时间;
8 2、索引需要占用物理空间
9 3、当对表进行增、删、改、的时候索引也要动态维护, 这样就降低了数据的维护速度
10
11 # 3. 建立原则
12 1、频繁用来查询的字段上建立索引

```

- 13 2、需要排序的字段上建立索引
- 14 3、频繁使用条件判断的字段建立索引

■ 常用存储引擎及特点

- 1 1、InnoDB
- 2 2、MyISAM
- 3 3、MEMORY

■ MySQL数据库的优化

- 1 1、存储引擎优化
- 2 2、索引优化
- 3 3、SQL语句优化

■ MySQL锁

- 1 1、锁类型分类：读锁、写锁
- 2 2、锁粒度分类：表级锁、行级锁

■ MySQL数据库重置密码

- 1 1、`sudo -i`
- 2 2、`/etc/init.d/mysql stop`
- 3 3、`cd /etc/mysql/mysql.conf.d`
- 4 4、`vi mysqld.cnf` 在 [mysqld]下添加如下语句后保存退出
- 5 `skip-grant-tables`
- 6 5、`/etc/init.d/mysql start`
- 7 6、命令行: `mysql`
- 8 7、切换库: `use mysql`
- 9 8、重置密码:
- 10 `update user set authentication_string=password('新密码') where user='root';`
- 11 9、去掉配置文件中刚添加的 `skip-grant-tables`
- 12 10、重启mysql服务登录 `/etc/init.d/mysql restart`

基础面试题

■ Python2和Python3的区别

- 1 # 1、编码格式区别
- 2 python2: 默认编码格式是ascii,如果代码中使用中文需加如下声明:
- 3 `#!/coding=utf-8 或者 #!-*- coding=utf-8 -*-`
- 4 python3: 默认编码为utf-8,并且引入了字节串
- 5
- 6 # 2、函数区别
- 7 1、print
- 8 python2中是关键字,`print "hello world"`
- 9 python3中是函数,`print()`
- 10 2、input
- 11 python2中有和raw_input()

```

12 python3中只有input()
13 3、range
14 python2中range得到列表,xrange得到迭代器
15 python3中只有range,得到迭代器
16
17 # 3、运算修改
18 python2中: 3/2=1 3.0/2=1.5
19 python3中: 3/2=1.5 3.0/2=1.5
20
21 # 4、数据类型
22 python3中引入字节串,python3废弃python2的long类型,只有int
23
24 # 5、语法变化
25 1、异常: python2中异常和异常对象用逗号隔开
26     except Exception,e
27 2、python3中更加严格的缩进规则,在python2中tab和对应数量空格等同,python3中则不可以

```

■ 深拷贝和浅拷贝

```

1 # 1、浅拷贝定义及特点
2 1、定义: 对另外一个变量的内存地址引用的拷贝, 这两个变量指向同一个内存地址的变量值
3 2、特点:
4     公用一个值
5     这两个变量的内存地址一样
6     对其中一个变量值改变, 另一个变量的值也会改变
7 # 2、深拷贝定义及特点
8 1、定义: 一个变量对另外一个变量的值拷贝
9 2、特点:
10    两个变量的内存地址不同
11    两个变量各有自己的值, 且互不影响
12    对其任意一个变量的值的改变不会影响另外一个
13
14 # 3、python中如何实现?
15 浅拷贝: copy.copy()
16 深拷贝: copy.deepcopy()
17
18 # 简单总结
19 1、拷贝使用Python标准库模块 copy 实现
20 2、copy.copy()内部拷贝了可变类型当时的引用,而copy.deepcopy()所有元素都拷贝

```

